

Project Design Phase-II

Technology Stack (Architecture & Stack)

Metadata

- **Date:** June 19, 2026
- **Team ID:** -----
- **Project Name:** AI-Enhanced Intrusion Detection System

1. Technical Architecture Diagram Description

The data flows sequentially across localized systemic layers to handle request isolation and feature categorization without external cloud dependence.

- **User/Admin Layer:** Interacts with the presentation layout built on HTML5, CSS3, and Bootstrap 5 utilities.
- **Application Core Block (Local System):** A Python 3 web server running Flask 3.1 that manages routing (app.py), captures multi-field connection features, and logs active alerts directly into volatile, runtime storage arrays.
- **Machine Learning Pipeline Interface:** The Application Core passes vectors via an internal function call into a scikit-learn processing pipeline. A serialized StandardScaler handles mathematical feature scaling, and an ensemble Random Forest Classifier executes multi-class voting over 100 decision trees.
- **Output / Application UI:** Returns a clean web interface view containing active alert states or broadcasts a programmatic JSON object payload to calling API scripts.

1. User Interface (The Screen You See)

- **What it is:** The visual layout of the website that you interact with. It contains the dashboard graphs, the text input fields, and the alert table logs.
- **The Tools:** **HTML5 & CSS3** build the structure and styling; **Bootstrap 5** makes sure it shrinks or expands nicely to fit both mobile screens and desktops; **Chart.js** draws the live, colorful metric charts.

2. Application Logic-1 (The Brains / Router)

- **What it is:** The central backend engine running behind the scenes (app.py). It acts like a traffic cop—taking your requests from the web browser, passing them to the AI model, and sending the results back to your screen.
- **The Tools:** **Python 3 & Flask 3.1** are used to build this fast, lightweight web server framework.

3. Application Logic-2 (The Training Setup)

- **What it is:** The preparation script (`train_model.py`) used to read the training dataset, clean up the mathematical columns, and teach the AI how to recognize different types of hacks before the app goes live.
- **The Tools: Pandas & NumPy** are standard Python toolkits used to sort, slice, and structure large tables of numbers easily.

4. Database (The Quick Memory)

- **What it is:** A simple, temporary system storage space that memorizes your ongoing attack logs during runtime so they can be viewed on the history table.
- **The Tools: Volatile Local Memory.** Instead of setting up a heavy external database software like MySQL, the app uses **Python's built-in memory lists**. This keeps things incredibly fast, though it clears out whenever you turn off the server.

5. File Storage (The Project Folder)

- **What it is:** The organized folder hierarchy on your computer's hard drive that holds your code sheets, layout templates, website images, and saved AI models.
- **The Tools: Local Filesystem Architecture** (your standard computer folders).

6. External API-1 (The Automated Communication Link)

- **What it is:** A background bridge path (`POST /api/predict`) that allows *other* computer programs or automated scripts to secretly send network numbers to your system and get back a flat answer without needing to load the human web dashboard.
- **The Tools:** A built-in **Flask REST Endpoint**.

7. Machine Learning Model (The Core Predictor)

- **What it is:** The actual AI engine. It looks at 10 numeric pieces of information from a connection and calculates whether it is completely safe or fits the profile of a malicious attack (like DoS or Probing).
- **The Tools: scikit-learn.** It uses a **Random Forest Classifier** (an ensemble of 100 decision trees voting together) and a **StandardScaler** to normalize incoming data points.

8. Model Storage Architecture (The Saved Brain File)

- **What it is:** A compressed file snapshot of your AI model after it finishes training. Instead of spending hours retraining the model every time you launch the web server, the app just opens this pre-saved file instantly.

- **The Tools: joblib or Pickle files (.pkl)** which freeze your trained machine learning pipelines into tiny, deployable binary files.

9. Infrastructure (Where It Runs)

- **What it is:** The actual server hosting the application.
- **The Tools: Local System Environment (<http://localhost:5000>)**. The full system runs locally on your own machine's processor without needing any paid, external cloud hosting accounts.

1. Open-Source Frameworks (Built with Free Tools)

- **What it means:** The entire project is built using free, community-driven software development tools instead of expensive, paid commercial software.
- **In simple terms:** It means anyone can download, run, or study your project code without needing to purchase a software license.
- **The Tools:** **Flask** acts as the web frame, **Python** provides the language, **scikit-learn** brings the AI logic, and **Git/GitHub** keeps track of code changes.

2. Security Implementations (How It Shields Itself)

- **What it means:** The application is built to prevent hacker intrusion or malicious crashes by locking itself down locally and checking incoming data.
- **In simple terms:** * It only runs inside your own computer (**Localhost Deployment**), meaning random people on the internet cannot access or attack your app.
 - It rejects corrupt files or broken traffic data by enforcing a strict check that demands exactly **10 specific network metrics** before it lets the AI process anything.

3. Scalability Architecture (Ready for Heavy Traffic)

- **What it means:** The code is split up so that the machine learning math engine runs completely separately from the visual web layout pages.
- **In simple terms:** The AI uses a "**Stateless Inference Layer**." This means every time data arrives, the AI instantly looks at it, outputs a prediction, and immediately forgets it. Because it doesn't hold onto heavy historical data weights during the math check, you can easily copy and run this engine on multiple servers to handle thousands of requests simultaneously without slowing down.

4. Availability (Runs Anywhere Easily)

- **What it means:** The app has a tiny digital footprint, meaning it doesn't require massive computer power, expensive hardware, or heavy graphics cards to run smoothly.
- **In simple terms:** It can easily run on a standard, low-power school laptop. All the rules and instructions the computer needs to start the server are stored in a tiny text file called **requirements.txt**.

5. Performance (Lightning Fast Speeds)

- **What it means:** The time it takes for a user to send network numbers to the AI and receive a safety verdict happens in less than a single millisecond (sub-millisecond speed).
- **In simple terms:** Because the system uses highly optimized **Random Forest decision trees** and keeps active data templates ready in the computer's temporary high-speed memory caches, the AI can make complex security decisions faster than a blink of an eye.